Cluster-Swap: Consistent Cross-View Poisoning for Untargeted Byzantine Attacks in Vertical Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Vertical Federated Learning (VFL) is a privacy-preserving collaborative training paradigm in which participants holding disjoint feature sets over a shared sample space jointly train a global model without exposing raw data. While the security of VFL has attracted growing research interest, the literature remains heavily skewed toward targeted threats, particularly backdoor and reconstruction attacks. Byzantine attacks, a family of untargeted poisoning threats that degrade global model accuracy, have been extensively studied in horizontal federated learning but remain largely unexplored in the vertical setting. This gap is especially consequential: the classical two-party design of many VFL systems allows a single compromised passive party to unilaterally destabilize the entire system.

In this paper, we introduce a novel Byzantine attack strategy for vertical settings that performs consistent cluster-based swapping within the adversary's local feature space by inferring latent class structure from limited supervision, constructing a persistent and statistically plausible poisoned cross-view association during training. The global model internalizes this misalignment and catastrophically fails on clean, correctly aligned data at inference time. We further demonstrate that state-of-the-art VFL defenses fail to detect this attack without incurring substantial performance penalties. To address this, we propose a targeted defense that attributes and repairs the corrupted associations via reference-guided reconstruction, restoring model performance without retraining from scratch. Our findings establish untargeted Byzantine attacks as a real, underappreciated availability threat to VFL and motivate robustness mechanisms tailored to its unique cross-view learning dynamics.

1 Introduction

Federated learning (FL) emerged from the need for multiple organizations to jointly train models without centralizing raw data, thereby reducing regulatory and operational barriers to data sharing [McMahan et al., 2023]. In horizontal FL (HFL), participants hold disjoint samples drawn from the same feature space, and the server aggregates local model updates. In contrast, vertical FL (VFL) partitions the feature space across parties that share a common entity index, for example, a bank and a telecom serving the same customers [Vepakomma et al., 2018]. VFL is increasingly used in high-value cross-silo settings such as credit scoring, risk modeling, and healthcare analytics [Yang et al., 2023], where no single party can legally or operationally hold all relevant attributes.

In VFL, each party's bottom model encodes its private feature slice into intermediate embeddings, which are forwarded to a server-side head model; the server, which typically holds the labels, returns gradients to the participating parties [Cheng et al., 2021]. This architecture creates security risks that differ fundamentally from HFL. Firstly, gradients and activations can leak the server's private

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

labels [Fu et al., 2022, He et al., 2024]. Secondly, a malicious party can corrupt the joint decision surface by submitting adversarial representations during training [Naseri et al., 2023, Chen et al., 2024]. The second risk is especially disruptive: unlike inference-time attacks that may be neutralized without retraining, training-time poisoning contaminates the learned parameters themselves. Once the server fits a global model on corrupted embeddings, recovery may require expensive retraining and prolonged loss of service [He et al., 2024, Gu and Bai, 2023].

Much of the existing VFL security literature has focused on targeted threats, including backdoor attacks, label inference, and reconstruction attacks [Naseri et al., 2023, Fu et al., 2022, Gu and Bai, 2023, Chen et al., 2024]. In parallel, the HFL literature studies *Byzantine* behavior, where a malicious client acts persistently and arbitrarily to degrade the global model [Fang et al., 2020]. Unlike classical backdoors, which aim to misclassify specific trigger-target pairs while preserving clean accuracy, Byzantine objectives are typically untargeted: the goal is to degrade overall model performance while remaining difficult to detect during training. Despite this relevance, untargeted Byzantine attacks remain underexplored in VFL, even though the classical two-party VFL design can allow a single compromised party to destabilize the entire system.

Why HFL defenses do not directly transfer. Byzantine defenses in HFL typically rely on update homogeneity: benign clients optimize over the same feature space and thus produce comparable updates, enabling robust aggregation or outlier filtering [Shi et al., 2022, So et al., 2020]. This assumption fails in VFL. Parties encode different feature spaces, and the server fuses heterogeneous representations rather than aggregating interchangeable model updates. As a result, an "outlier among models" is structurally ill-defined, and similarity-based filtering may penalize legitimate feature heterogeneity. Existing VFL defenses therefore monitor local signals such as smashed-feature distributions, gradient norms, temporal drift, or reconstruction error, but these signals capture batch-level or party-level abnormalities. They are not designed to detect attacks that remain locally plausible while corrupting the global cross-view association learned by the head model.

Byzantine attacks in VFL: practical gaps. Byzantine robustness in VFL remains far less explored than in HFL. Existing VFL studies primarily adapt defenses to HFL-inspired or relatively direct Byzantine behaviors, including arbitrary corrupted values, message tampering, label-flipping, bit-flipping, and sign-flipping attacks [Yuan et al., 2022, Xu et al., 2024, Oonishi and Nakai, 2025]. These attacks are useful stress tests, but they frame Byzantine behavior as a local abnormality in transmitted values or client contributions. VFL, however, is structurally different: parties encode heterogeneous feature spaces, and the server learns from their fused cross-party representation rather than aggregating comparable model updates. This heterogeneity is known to undermine HFL-style similarity and outlier defenses [Xu et al., 2024, Oonishi and Nakai, 2025], but its offensive implication has been largely overlooked. A malicious party can submit statistically plausible features while corrupting the global association between views. We study this overlooked attack surface and show that consistent cross-view poisoning can degrade VFL availability without transport-channel tampering or test-time triggers.

Contribution. We introduce *Consistent Cross-View Poisoning*, a training-phase untargeted Byzantine attack tailored to VFL. Our central insight is that VFL's strength, learning from joint cross-party representations, also creates a structural attack surface. Concretely, we contribute:

- **We identify a new availability threat in VFL.** Unlike the prior studies on VFL focusing on targeted attacks, we introduce a novel attack surface by formalizing an untargeted Byzantine threat that degrades the overall clean accuracy of the deployed VFL model without requiring test-time triggers or target labels.
- **We show how cross-view alignment can be systematically poisoned.** Our attack corrupts the VFL training through consistent cluster-level swaps. This makes the server learn stable but false cross-view associations while never requiring access to the honest party's features.
- **We explain why existing defenses miss this failure mode.** We show that many VFL and HFL-inspired defenses monitor local abnormalities, such as gradient norms, embedding drift, or reconstruction error. CCVS shows detect-and-drop mitigation unreliable and sometimes utility-destroying in limited party settings like VFL systems

- **We introduce a repair-based defense for corrupted representations** to detect cross-view inconsistency, attribute the corrupted party, and reconstruct a plausible honest representation to save the model from catastrophic failure from Byzantine threats.

2 Preliminaries

2.1 Vertical Federated Learning

Let K clients be indexed by $c \in \{1, \dots, K\}$ over N aligned samples. For sample i, client c holds a private feature view $x_i^{(c)} \in \mathbb{R}^{d_c}$, while the full feature vector $x_i = \{x_i^{(1)}, \dots, x_i^{(K)}\}$ remains distributed across parties. Each client encodes its local view through a bottom model $h_i^{(c)} = f_c(x_i^{(c)}; \theta_c)$. The server or active party concatenates the embeddings and applies a head model:

$$\hat{y}_i = g\Big(\|_{c=1}^{K} h_i^{(c)}; \theta_g\Big).$$

Only the server or active party holds labels $y_i \in \mathcal{Y}$, and the system is trained end-to-end by minimizing

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{y}_i, y_i), \qquad \Theta = \{\theta_1, \dots, \theta_K, \theta_g\}.$$

During back-propagation, the server sends each client $\nabla_{h_i^{(c)}} \mathcal{L}$, enabling local bottom-model updates without exchanging raw features. Thus, VFL learns over the joint feature space while exposing only intermediate representations and gradients.

2.2 Byzantine Attacks in VFL

A Byzantine attack is an adversarial deviation by a subset of malicious clients $\mathcal{B} \subset \{1, \dots, K\}$ to corrupt the global model. In HFL, Byzantine clients typically manipulate local model updates before aggregation. In VFL, clients instead submit intermediate representations from heterogeneous feature spaces. A malicious client can therefore corrupt training by poisoning its local input view or by perturbing its transmitted embedding:

$$\tilde{h}_i^{(c)} = f_c(\tilde{x}_i^{(c)}; \theta_c) \quad \text{or} \quad \tilde{h}_i^{(c)} = \psi\Big(f_c(x_i^{(c)}; \theta_c)\Big), \qquad c \in \mathcal{B},$$

where $\tilde{x}_i^{(c)}$ is a crafted input and $\psi(\cdot)$ is an embedding-level transformation. For benign clients $c \notin \mathcal{B}$, the clean embedding $h_i^{(c)}$ is used. The server trains on the mixed representation

$$\tilde{H}_i = \|_{c=1}^{K} \begin{cases} \tilde{h}_i^{(c)}, & c \in \mathcal{B}, \\ h_i^{(c)}, & c \notin \mathcal{B}, \end{cases} \qquad \hat{y}_i = g(\tilde{H}_i; \theta_g).$$

The threat considered here is *untargeted*: unlike backdoors, which aim for trigger-specific misclassification while preserving clean accuracy, the adversary seeks to degrade clean inference-time performance after training. This is difficult to defend against in VFL because HFL-style defenses rely on homogeneous client updates, whereas VFL parties encode different feature spaces by construction.

2.3 Threat Model

Our threat model follows all the classic VFL protocols and carries out most experiments in the classic two-party ($K = 2$) VFL setting, the dominant real-world deployment [Yang et al., 2023, Vepakomma et al., 2018] and the adversarially worst case: the malicious party controls half the joint representation. The adversarial party A holds $X^{(A)} \in \mathbb{R}^{N \times d_A}$ and only has access to their own bottom model f_A. Party A cannot access the feature-set or local model of other benign parties, label $\mathbf{y}$, or any message between the server and benign parties. Party A additionally holds a small stratified auxiliary set

$$\mathcal{D}_L = \big\{(x_i^{(A)},\, y_i)\big\}_{i \in \mathcal{I}_L}, \quad |\mathcal{I}_L| = \lfloor \varepsilon N \rfloor, \quad \varepsilon \in [0.02,\, 0.03], \tag{1}$$

with at least one sample per class. This is strictly weaker than full label access, and is consistent with realistic partial-label leakage scenarios in VFL [Fu et al., 2022, He et al., 2024], contractual outcome sharing between collaborating institutions, and partially public outcome records.

Adversarial objective. Party A replaces its training features with a rearranged set $\widetilde{X}^{(A)}$, where $\widetilde{x}_i^{(A)} = x_{j(i)}^{(A)}$ for some $j(i) \in [N]$. The server, oblivious to the poisoning, fits:

$$g^\star = \operatorname*{argmin}_g \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\big(g\big(\widetilde{h}_i^{(A)} \| \, h_i^{(B)}\big),\, y_i\big), \quad \widetilde{h}_i^{(A)} = f_A(\widetilde{x}_i^{(A)}). \tag{2}$$

$$\max_{\widetilde{X}^{(A)}} \mathbb{E}_{P_{\text{clean}}}\Big[\mathcal{L}\big(g^\star\big(f_A(x^{(A)}) \| \, f_B(x^{(B)})\big),\, y\big)\Big], \tag{3}$$

The attacker maximises the *clean-test* loss of $g^\star$ forcing $g^\star$ to internalize a spurious cross-view association that collapses at inference on clean, correctly aligned data.

RGAR: one-time reference setup. RGAR requires the server to hold a small statically authenticated reference set before training begins:

$$\mathcal{R} \subset [N], \quad |\mathcal{R}| = \lfloor r_{\text{ref}} N \rfloor, \quad r_{\text{ref}} \in [0.05,\, 0.10], \quad |\mathcal{R} \cap \{i : y_i = c\}| \geq 1 \;\; \forall\, c \in \mathcal{Y}. \tag{4}$$

Party A submits $\{x_i^{(A)}\}_{i \in \mathcal{R}}$ to the server once via an authenticated channel; the server stores an immutable copy and no labels are disclosed to Party A. We adopt the conservative assumption that Party A knows $\mathcal{R}$; since it cannot modify the server's authenticated copy, RGAR remains effective. This single pre-training exchange is operationally comparable to the pre-training entity alignment mandated in production VFL systems [Yu et al., 2024], and provides the calibration anchor for RGAR's scoring, attribution, and reconstruction stages (§4).

3 Methodology

To achieve the Byzantine objective, we design a attack pipeline with two stages. The first stage constructs high-fidelity clusters of the training samples without observing the true labels. In the second stage, these inferred clusters guide a consistent feature-swapping attack in the training phase that distorts the association between feature subsets during vertical training.

3.1 Phase I: Semi-supervised Clustering via Latent Representation

Phase I infers a latent partition of the adversary's local feature view. Given the adversarial training view $X^{(A)} = \{x_i^{(A)}\}_{i=1}^N$ and a small auxiliary labeled subset $\mathcal{D}_L$, the adversary learns an embedding function and outputs pseudo-cluster assignments and confidence scores.

$$\phi : \, x_i^{(A)} \mapsto z_i \in \mathbb{R}^p, \qquad \hat{c}_i \in \{1, \ldots, K_c\}, \qquad \hat{q}_i \in [0, 1].$$

The goal is to recover enough class-correlated structure from the adversary's own view so that Phase II can construct semantically misleading class-consistent cross-view pairs.

Representation learning. Because the adversary observes only a partial vertical view, Phase I uses modality-matched representation learning. For vision datasets, we learn semantic embeddings with contrastive learning. An encoder is first trained with the NT-Xent objective used in SimCLR [Chen et al., 2020]:

$$\mathcal{L}_{\text{con}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}.$$

For grayscale datasets, we further refine the embedding space on $\mathcal{D}_L$ using supervised contrastive learning [Khosla et al., 2020]:

$$\mathcal{L}_{\text{sup}} = -\sum_{i \in \mathcal{D}_L} \frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \neq i} \exp(\text{sim}(z_i, z_a)/\tau)},$$

where $\mathcal{P}(i)$ is the set of labeled positives for sample i.

For harder half-view or tabular settings, we use a FixMatch-style student–teacher pipeline [Sohn et al., 2020]. The student learns from $\mathcal{D}_L$ and high-confidence pseudo-labels, while an EMA teacher provides stable predictions:

$$\theta_t \leftarrow m\theta_t + (1-m)\theta_s, \qquad \hat{c}_i = \arg\max_y p_{\theta_t}(y \mid x_i^{(A)}), \quad \hat{q}_i = \max_y p_{\theta_t}(y \mid x_i^{(A)}).$$

Table 1 summarizes the Phase I configuration for all six datasets.

Table 1: Phase I configuration summary. "Adversarial view" describes the local feature view available to the malicious party; 2-3% auxiliary labeled fraction used only for local structure inference; K_c is the number of clusters. GMM means Gaussian mixture models.

Dataset	Adversarial view	K_c	Pipeline
MNIST	Left 14 cols	10	SimCLR → SupCon → GMM merge
Fashion-MNIST	Left 14 cols	10	SimCLR → SupCon → GMM merge
CIFAR-10	Left 16 RGB cols	10	SimCLR → FixMatch → GMM/teacher
UCI-HAR	Top-MI continuous block	6	Tabular FixMatch
UCI-Mushroom	Top-MI discrete block	2	Bernoulli mixture + pseudo-label refinement
UCI-Bank	Skewed MI-ranked block	16	Attack-oriented GMM[†]

[†]For UCI-Bank, $K_c = 16$ is chosen to increase geometric donor diversity for Phase II.

Clustering and confidence. For mixture-based pipelines, embeddings are L2-normalized and clustered using a Gaussian or Bernoulli mixture model [McLachlan and Peel, 2000], depending on the modality. The cluster identity is the maximum-responsibility component after any prototype-based merging, and the confidence score is the corresponding soft membership probability:

$$\hat{c}_i = \arg\max_k r_{ik}, \qquad \hat{q}_i = \max_k r_{ik}.$$

These assignments and confidence scores are passed to Phase II, yielding clusters that closely mirror the unknown label partition and thus supply a high-quality surrogate for guiding the subsequent class-consistent view–swapping attack.

3.2 Phase II: Class-Consistent View Swap

Intuition and training-time mechanism. Guided by the predicted clusters from Phase I, the adversary constructs a poisoned training set by consistently exchanging its local feature view across clusters. The core intuition is that VFL relies on stable cross-party correlations between the feature views of each aligned entity. If these correlations are systematically disrupted during training, the server-side head model is encouraged to learn spurious cross-view associations that do not generalize to clean data.

Core mechanism. Let $\mathcal{G} = \{C_1, \ldots, C_{K_c}\}$ be the cluster partition from Phase I and $c(i) \in [K_c]$ sample i's assignment. For each source cluster C_s, the adversary identifies a disjoint destination cluster $C_{t(s)}$ and replaces every victim $i \in C_s$ with a *donor* sample $j(i) \in C_{t(s)}$:

$$\widetilde{x}_i^{(A)} \leftarrow x_{j(i)}^{(A)}, \quad j(i) \in C_{t(c(i))}, \quad \forall\, i \in [N]. \tag{5}$$

Server labels y_i and Party B's features $x_i^{(B)}$ remain unchanged. The server therefore trains on triples $(\widetilde{x}_i^{(A)}, x_i^{(B)}, y_i)$ whose two feature views are drawn from different class-conditioned manifolds, internalizing a stable but semantically inverted cross-view association. At inference, the clean pair $(x_i^{(A)}, x_i^{(B)})$ violates this learned pattern, causing marked accuracy collapse.

Why cluster-level consistency is essential. Independent per-sample noise averages out over mini-batches, yielding negligible stable signal. Our cluster-wise permutation applies the *same* source-to-destination mapping for every sample in a cluster across all training epochs, maximising the batch-level mutual information between the fabricated pair and the label:

$$\mathrm{MI}\big(\widetilde{X}^{(A)}, X^{(B)}\big) \gg \mathrm{MI}\big(X_{\text{noise}}^{(A)}, X^{(B)}\big), \tag{6}$$

where $X_{\text{noise}}^{(A)}$ denotes random per-sample corruption. This ensures the server reliably fits the fabricated cross-view association rather than treating it as noise.

Optimal cluster-to-cluster mapping. We formalize the cluster mapping as a derangement $\pi \in \mathfrak{D}_{K_c}$, so no cluster maps to itself. CCVS selects the mapping that maximizes inter-cluster distance:

$$\pi^\star \in \arg\max_{\pi \in \mathfrak{D}_{K_c}} \sum_{s=1}^{K_c} D_{s,\pi(s)}, \qquad D_{st} = 1 - \langle \mu_s, \mu_t \rangle, \tag{7}$$

where $\mu_s = {}^1/_{|C_s|} \sum_{i \in C_s} v_i$ is the L2-normalized centroid of cluster C_s in the attacker representation space. This global mapping pairs each source cluster with a geometrically distant destination, maximizing semantic mismatch.

Given $\pi^\star$, each victim $i \in C_s$ receives a hardest-negative donor from the top-q confidence core of the destination cluster:

$$j^\star(i) = \arg \min_{j \in \mathcal{D}^{(q)}_{\pi^\star(s)}} \langle v_i, v_j \rangle, \qquad \mathcal{D}^{(q)}_t = \{j \in C_t : \hat{q}_j \geq \mathrm{quant}(\hat{q}_{C_t}, q)\}. \tag{8}$$

Greedy diversity cycling prevents a small set of donors from dominating by using all samples in the high-confidence core before repetition. Thus, CCVS combines a *global* maximum-distance cluster mapping with a *local* hardest-negative donor assignment.

Why the optimal swap is strongest. The optimal swap creates a high-contrast, internally consistent, but incorrect cross-view relation. Random sample-level corruption often behaves like noise and can be averaged out across mini-batches. Cluster-level swapping instead repeats the same source-to-destination relation throughout training, giving the server a stable but semantically inverted signal: the adversarial view remains locally valid, but its association with the honest view and label is wrong.

Consequently, the head model is encouraged to fit the fabricated cross-view alignment. At inference time, clean test pairs are correctly aligned and no longer follow the reinforced cross-cluster pattern, causing the learned decision rule to fail. The attack therefore requires no test-time trigger or direct label manipulation; all damage comes from training-time corruption of feature alignment. Algorithm 1 details the full two-phase pipeline.

Algorithm 1 CCVS — Two-Phase Cluster-Swap Attack on Party A's View

Require: Party A's features $\{x_i^{(A)}\}_{i=1}^N$; auxiliary set $\mathcal{D}_L$ ($\approx 2\%$–3% labelled); modality $\mathcal{M}$; confidence quantile q; top-k donor scope.

Ensure: Poisoned training view $\{\widetilde{x}_i^{(A)}\}_{i=1}^N$.

// Phase I: Semi-supervised clustering
Extract embeddings via modality pipeline $V \leftarrow \mathrm{Embed}_{\mathcal{M}}(X^{(A)}, \mathcal{D}_L)$.
Fit clusters $\{C_s\}_{s=1}^{K_c}$ and confidences $\hat{q}$ from V.
Compute centroids: $\mu_s \leftarrow \frac{1}{|C_s|} \sum_{i \in C_s} v_i \,/\, \| \cdot \|, \forall s$.
// Phase II: Maximum-distance cluster swap
Build distance matrix $D_{st} = 1 - \langle \mu_s, \mu_t \rangle$.
Solve $\pi^\star \in \mathrm{argmax}_{\pi \in \mathfrak{D}_{K_c}} \sum_s D_{s,\pi(s)}$ via Eq. (7).
for $s = 1$ to K_c **do**
    Union top-k donor cores: $U_s = \bigcup_{t \in \mathcal{T}(s)} \mathcal{D}_t^{(q)}$, where $\mathcal{T}(s)$ are the k farthest clusters.
    for each victim $i \in C_s$ **do**
        $j^\star(i) = \mathrm{argmin}_{j \in U_s,\, j \text{ unused}} \langle v_i, v_j \rangle$ (Eq. (8)).
        $\widetilde{x}_i^{(A)} \leftarrow x_{j^\star(i)}^{(A)}$.
    end for
end for
return $\{\widetilde{x}_i^{(A)}\}_{i=1}^N$.

4 Defense

RGAR: Reference-Guided Attribution and Reconstruction. Many VFL and HFL-inspired Byzantine defenses follow a *detect-and-drop* principle: suspicious embeddings or updates are discarded, clipped, or zeroed, as in robust aggregation, trust-filtering, and reconstruction-based defenses [Blanchard et al., 2017, Cao et al., 2021b, Cho et al., 2024]. In VFL, this can be self-defeating because number of parties in real-life VFL systems are much lower than HFL systems. Suppressing one party in a few-party settings may remove a substantial part of the joint representation. RGAR instead detects unreliable cross-view pairs, attributes fault to a party, and repairs the corrupted view rather than discarding it.

Setup and reference set. RGAR uses the trusted calibration set $\mathcal{R}$ from Eq. (4). Before training, the server obtains an authenticated clean reference copy for the selected indices and forms reference triples $(h_i^A, h_i^B, y_i)_{i\in\mathcal{R}}$ using its own labels. This set calibrates clean cross-view geometry, trains the reconstructor, and scores training-time deviations; no server labels are disclosed to the passive party.

Stage A: Reference modeling. Using $\mathcal{R}$, the server estimates class prototypes $\mathbf{p}_c^A, \mathbf{p}_c^B$, diagonal Mahalanobis variances [Mahalanobis, 1936], and joint prototypes $\mathbf{p}_c^{\text{int}}$ over concatenated embeddings to model clean party-specific and cross-view geometry. It also trains a frozen reconstructor $g_\theta(h_i^B, y_i) \to \hat{h}_i^A$ using SmoothL1 for robustness to large residuals [Girshick, 2015] :

$$\mathcal{L}_{\text{recon}} = \frac{1}{|\mathcal{R}|} \sum_{i\in\mathcal{R}} \text{SmoothL1}\big(g_\theta(h_i^B, y_i), h_i^A\big), \tag{9}$$

Stage B: Online suspicion scoring. For each sample i, RGAR computes

$$s_i^{\text{proto}} = \alpha \cdot \frac{\bar{d}_i^A + \bar{d}_i^B}{2} + (1-\alpha) s_i^{\text{int}}, \tag{10}$$

$$s_i^{\text{int}} = 1 - \cos\Big(\frac{[h_i^A; h_i^B]}{\|[h_i^A; h_i^B]\|}, \mathbf{p}_{y_i}^{\text{int}}\Big), \tag{11}$$

where $\bar{d}_i^k$ is the scaled diagonal Mahalanobis distance to $\mathbf{p}_{y_i}^k$. This joint term detects cross-view inconsistency even when each view remains locally plausible. Party evidence e_i^A, e_i^B further adds temporal drift $1 - \cos(h_i^k, \text{EMA}_i^k)$ using exponential moving averages [Tarvainen and Valpola, 2017].

Stage C: Delayed attribution. For samples with $s_i^{\text{proto}} > \tau_{\text{pair}}$, RGAR accumulates the epoch-level signed evidence $e_i^A - e_i^B$. After burn-in, if its mean exceeds τ_{global}, the server attributes corruption to party A and decays $\rho_A \to \rho_{\min}$; attribution is revocable if later evidence reverses.

Stages D–E: Repair and reweighting. After attribution, RGAR constructs and repairs the live embedding by

$$\begin{aligned} \hat{h}_i^A &= \beta\, g_\theta(h_i^B, y_i) + (1-\beta)\mathbf{p}_{y_i}^A, \\ \tilde{h}_i^A &= (1-w_i) h_i^A + w_i \hat{h}_i^A. \end{aligned} \tag{12}$$

where w_i grows with s_i^{proto} and increases after attribution. Party B remains trusted with $\rho_B \approx 1$. For asymmetric architectures, RGAR can fall back to $\hat{h}_i^A = \mathbf{p}_{y_i}^A$ and freeze the suspected encoder. Algorithm 2 in Appendix F summarizes the pipeline.

5 Experimental Evaluation

We evaluate CCVS and RGAR along four questions: (i) whether the adversary can infer useful latent partitions from only its local view, (ii) whether consistent cross-view poisoning degrades clean inference accuracy, (iii) whether standard VFL and HFL-inspired defenses detect or recover from the attack, and (iv) how the attack scales with party count and auxiliary label access.

5.1 Experimental Setup

Datasets and partitions. We evaluate on six datasets across vision, sensor, and tabular modalities: MNIST [LeCun et al., 1998], Fashion-MNIST [Xiao et al., 2017], CIFAR-10 [Krizhevsky, 2009], UCI-HAR [Anguita et al., 2013], UCI-Mushroom [Schlimmer, 1987], and UCI-Bank [Moro et al., 2014]. Images are vertically split at the mid-column. For tabular datasets, the adversarial view is built from mutual-information-ranked features estimated using only the auxiliary labeled subset. HAR and Mushroom use top-MI blocks, while Bank uses a skewed MI block to preserve donor diversity. Details are in Appendix A.

Models and training. MNIST and Fashion-MNIST use lightweight local encoders with a two-layer MLP server head. CIFAR-10 uses ResNet-based bottom models [He et al., 2016] with concatenation fusion. The tabular datasets use MLP-based VFL models, with asymmetric passive dimensions for HAR and Bank. Details are in Appendix B.

Attack strategies. We compare five poisoning strategies: **Optimal Topk**, our full CCVS method using maximum-distance derangement and hardest-negative donors; **Derangement**, which uses the same cluster mapping without hard donor selection; **Round-Robin**, a fixed cyclic cluster mapping; **Random Cluster**, a uniform random derangement; and **Random Sample**, which draws independent donors from the full training set. All preserve honest-party features and server labels.

Defense baselines. We compare RGAR with three detect-and-drop baselines: **Batch Krum**, adapted from Krum Byzantine-robust aggregation [Blanchard et al., 2017]; **Cosine Gate**, inspired by FLTrust's trusted-reference cosine scoring [Cao et al., 2021a]; and **AE Gate**, inspired by reconstruction-based VFL defense [Cho et al., 2024]. These baselines suppress suspicious embeddings rather than repair them.

Metrics. Phase I quality is measured by Hungarian accuracy (H-ACC) using optimal assignment [Kuhn, 1955], normalized mutual information (NMI), adjusted Rand index (ARI) [Hubert and Arabie, 1985], and purity. Attack strength is measured by clean-test accuracy after poisoned VFL training, where lower accuracy means stronger attack success rate. Defense performance is measured by post-defense clean-test accuracy and detection or suspicion rate.

Computation. All experiments are run on single-GPU or CPU setups without distributed training; Appendix G reports hardware, runtime, and total compute estimates.

5.2 Phase I: Latent Partition Quality

Table 2 shows that Phase I recovers useful latent partitions from the adversary's local view using only limited auxiliary labels. The clearest datasets show strong alignment, with UCI-Mushroom reaching H-ACC 0.978 and MNIST/UCI-HAR reaching H-ACC $0.881/0.874$. More complex datasets remain sufficiently structured for poisoning: Fashion-MNIST and CIFAR-10 reach H-ACC 0.751 and 0.723, respectively. Per-cluster size, majority label, and purity are reported in Appendix C.

Table 2: Phase I clustering quality on the training split. H-ACC: Hungarian accuracy; NMI: normalized mutual information; ARI: adjusted Rand index; Purity: dominant-label fraction per cluster.

Dataset	N	K_c	H-ACC	NMI	ARI	Purity
MNIST	60,000	10	0.881	0.787	0.756	0.881
Fashion-MNIST	60,000	10	0.751	0.699	0.610	0.751
CIFAR-10	50,000	10	0.723	0.544	0.507	0.723
UCI-HAR	8,754	6	0.874	0.787	0.743	0.874
UCI-Mushroom	6,905	2	0.978	0.861	0.913	0.978
UCI-Bank	36,168	16	0.914	0.812	0.795	0.883

5.3 Phase II: Attack Effectiveness

Table 3 shows that CCVS causes large clean-test accuracy drops across all six datasets in the two-party setting with 100% swap coverage. The strongest variants reduce accuracy by 37.0–98.3 pp without any test-time trigger. Results also show that random corruption is insufficient, while the largest drops arise from cluster-consistent, distance-aware swaps with hard donor selection.

Table 3: Post-attack global model accuracy (%) on clean test data after training on poisoned data. Lower is stronger. Bold indicates the strongest attack per dataset.

Dataset	Clean	Opt. Topk	Deranged	Round Robin	Rnd. Cluster	Rnd. Sample	Best Drop
MNIST	96.9	**33.0**	78.9	82.0	75.0	89.0	63.8
Fashion-MNIST	87.2	**33.6**	68.5	67.1	78.0	81.6	53.6
CIFAR-10	89.2	**52.2**	82.8	80.5	81.5	83.2	37.0
UCI-HAR	98.3	15.5	**0.0**	43.2	44.9	56.3	**98.3**
UCI-Mushroom	98.4	**26.3**	94.7	95.2	94.7	91.0	72.1
UCI-Bank	89.6	**49.0**	89.5	84.6	83.8	86.4	40.6

Table 4: Defense comparison on clean-test accuracy (%) and detection rate (%). RGAR repairs corrupted representations, while baselines suppress flagged embeddings. *Cosine accuracy for UCI-Bank reflects majority-class signal in the honest party's 2-D embedding, not genuine attack repair.

Dataset	Clean	No Defense	Krum		Cosine Gate		AE Gate		RGAR (ours)	
			Acc.	Det.	Acc.	Det.	Acc.	Det.	Acc.	Susp.
MNIST	96.8	33.3	38.2	6	46.9	100	33.6	3	**83.5**	100
Fashion-MNIST	87.2	32.2	41.5	4	13.3	100	32.9	18	**73.6**	100
UCI-HAR	98.3	15.5	15.5	5	61.2	97	29.1	10	**89.4**	100
UCI-Bank	89.6	49.0	32.7	4	88.3*	57	38.2	4	**85.3**	97
CIFAR-10	89.2	52.2	47.6	9	41.4	64	36.8	12	**84.7**	94
UCI-Mushroom	98.4	26.3	30.7	7	63.2	78	22.4	8	**92.4**	100

5.4 Defense Evaluation

Table 4 shows that detect-and-drop baselines either miss CCVS or recover by discarding useful signal. Krum and AE Gate detect few poisoned samples, indicating that CCVS remains locally plausible. Cosine Gate detects many samples, but zeroing the suspected view can collapse utility, as in Fashion-MNIST. This shows that even if a state-of-the-art defense detects our attack, it fails to save the global model performance. RGAR consistently provides the strongest genuine recovery by repairing the corrupted representation instead of suppressing it, improving accuracy from $33.3\% \rightarrow 83.5\%$ on MNIST, $32.2\% \rightarrow 73.6\%$ on Fashion-MNIST, $15.5\% \rightarrow 89.4\%$ on UCI-HAR, and $49.0\% \rightarrow 85.3\%$ on UCI-Bank. This supports our core defense claim: VFL needs representation repair, not representation removal.

5.5 Ablation Studies

Number of parties. Attack strength decreases as K increases because one Byzantine party controls a smaller share of the fused representation. This supports our focus on the classical two-party VFL setting, where a single compromised party has maximum leverage.

Auxiliary label fraction. CCVS remains effective with only 1–2% auxiliary labels, showing that limited supervision is enough to infer useful latent structure. Higher auxiliary fractions improve cluster fidelity and further strengthen poisoning. The ablation tables are reported in Appendix F.2.

6 Limitations

Our attack is most relevant and destructive in the classical two-party VFL setting, where a single compromised party has the highest leverage over the fused representation; extending CCVS to coordinated multi-party collaborative Byzantine adversaries is an important future direction. Our defense RGAR relies on a clean or independently verifiable reference set $\mathcal{R}$; if an adversary controls the passive party before calibration and can poison $\mathcal{R}$, the learned prototypes and reconstructor may be corrupted. This could be mitigated through TEE-backed feature extraction, frozen reference embeddings, or robust prototype filtering.

7 Conclusion

In this paper, we explore a novel gap in VFL security: untargeted Byzantine attacks that degrade model availability rather than implanting backdoors or inferring private labels. We introduce CCVS, a two-phase attack that infers latent representations and uses consistent cluster-level swaps to poison the cross-view associations learned by the server. Across six datasets, CCVS causes large clean-accuracy drops without any test-time trigger, showing that VFL's joint representation is itself a structural attack surface. We further show that detect-and-drop defenses are often insufficient and can remove useful joint signals. To address this, we propose RGAR, a repair-based defense that scores cross-view inconsistency, attributes the corrupted party, and reconstructs a plausible honest representation. Together, CCVS and RGAR establish cross-view alignment as a central object for both attacking and defending VFL systems, motivating robustness mechanisms that reason about global representation consistency in VFL systems.

References

Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge Luis Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *European Symposium on Artificial Neural Networks*, 2013.

Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021a.

Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021b.

Peng Chen, Xin Du, Zhihui Lu, and Hongfeng Chai. Universal adversarial backdoor attacks to fool vertical federated learning. *Computers & Security*, 137:103601, 2024.

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 1597–1607, 2020.

Kewei Cheng, Tao Fan, Yilun Jin, Yang Liu, Tianjian Chen, Dimitrios Papadopoulos, and Qiang Yang. Secureboost: A lossless federated learning framework, 2021. URL `https://arxiv.org/abs/1901.08755`.

Youngeun Cho et al. VFLIP: A backdoor defense for vertical federated learning. *arXiv preprint arXiv:2408.15591*, 2024.

Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. Local model poisoning attacks to {Byzantine-Robust} federated learning. In *29th USENIX security symposium (USENIX Security 20)*, pages 1605–1622, 2020.

Chong Fu, Xuhong Zhang, Shouling Ji, Jinyin Chen, Jingzheng Wu, Shanqing Guo, Jun Zhou, Alex X Liu, and Ting Wang. Label inference attacks against vertical federated learning. In *31st USENIX security symposium (USENIX Security 22)*, pages 1397–1414, 2022.

Ross Girshick. Fast R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1440–1448, 2015.

Yuhao Gu and Yuebin Bai. Lr-ba: Backdoor attack against vertical federated learning using local latent representations. *Computers & Security*, 129:103193, 2023.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Ying He, Zhili Shen, Jingyu Hua, Qixuan Dong, Jiacheng Niu, Wei Tong, Xu Huang, Chen Li, and Sheng Zhong. Backdoor attack against split neural network-based vertical federated learning. *IEEE Transactions on Information Forensics and Security*, 19:748–763, 2024. doi: 10.1109/TIFS.2023.3327853.

Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2:193–218, 1985.

Prannay Khosla, Pang Wei Tian, Chen Wang, Aaron Cartwright, Phillip Isola, Dilip Krishnan, Francois Barbalat, Ling Liu, Dhruv Krishnamurthy, Joshua Chang, Yuandong Tian, Phillip Isola, Dilip Krishnan, Carl Vondrick, Deva Ramanan, Cordelia Schmid, and Yonglong Tian. Supervised contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 18661–18673, 2020.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

Geoffrey McLachlan and David Peel. *Finite Mixture Models*. Wiley, 2000.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data, 2023. URL `https://arxiv.org/abs/1602.05629`.

Sérgio Moro, Paulo Cortez, and Paulo Rita. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62:22–31, 2014.

Mohammad Naseri, Yufei Han, and Emiliano De Cristofaro. Badvfl: Backdoor attacks in vertical federated learning, 2023. URL `https://arxiv.org/abs/2304.08847`.

Kento Oonishi and Tsunato Nakai. CC-VFed: Client contribution detects byzantine attacks in vertical federated learning, 2025. URL `https://openreview.net/forum?id=E3qIInyTgL`.

Jeff Schlimmer. Mushroom dataset. UCI Machine Learning Repository, 1987.

Junyu Shi, Wei Wan, Shengshan Hu, Jianrong Lu, and Leo Yu Zhang. Challenges and approaches for mitigating byzantine attacks in federated learning. In *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 139–146. IEEE, 2022.

Jinhyun So, Başak Güler, and A Salman Avestimehr. Byzantine-resilient secure federated learning. *IEEE Journal on Selected Areas in Communications*, 39(7):2168–2181, 2020.

Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 596–608, 2020.

Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems*, 2017.

Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data, 2018.

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

Jiuyun Xu, Yinyue Jiang, Hanfei Fan, and Qiqi Wang. Svfldetector: a decentralized client detection method for byzantine problem in vertical federated learning. *Computing*, 106(5):1659–1679, 2024.

Liu Yang, Di Chai, Junxue Zhang, Yilun Jin, Leye Wang, Hao Liu, Han Tian, Qian Xu, and Kai Chen. A survey on vertical federated learning: From a layered perspective, 2023. URL `https://arxiv.org/abs/2304.01829`.

Lei Yu, Meng Han, Yiming Li, Changting Lin, Yao Zhang, Mingyang Zhang, Yan Liu, Haiqin Weng, Yuseok Jeon, Ka-Ho Chow, and Stacy Patterson. A survey of privacy threats and defense in vertical federated learning: From model life cycle perspective, 2024. URL `https://arxiv.org/abs/2402.03688`.

Kun Yuan, Zhaoxian Wu, and Qing Ling. A byzantine-resilient dual subgradient method for vertical federated learning. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4273–4277. IEEE, 2022.

A Phase I Pipeline Details

A.1 Vertical Partitioning

Vision datasets. All image datasets are split along the pixel-width axis to match the standard split-view VFL geometry. For MNIST and Fashion-MNIST, Party A receives the left 14 grayscale columns and Party B receives the right 14 columns. For CIFAR-10, Party A receives the left 16 RGB columns and Party B receives the right 16 RGB columns. RGB images are un-normalized to $[0, 1]$ before contrastive augmentations.

UCI-HAR. Features are ranked by mutual information $\widehat{\mathrm{MI}}(x_d; y)$ estimated on the auxiliary labeled subset. Party A receives the top $\eta_A = 0.88$ fraction of features, approximately 477 dimensions, and Party B receives the remaining features. The same ranking is used for Phase I clustering and Phase II swapping.

UCI-Mushroom. The same MI-ranking strategy is applied to the binary and one-hot encoded features. Party A receives the top-MI feature block, and Party B receives the remaining features.

UCI-Bank. For the attack pipeline, Party A receives a skewed MI-ranked share ($\eta_A = 0.92$), concentrating high-information features on the adversarial side. For clean baseline training, a balanced round-robin feature split is used.

A.2 Modality-Matched Clustering Pipelines

Grayscale vision: MNIST and Fashion-MNIST. For grayscale vision datasets, we first pretrain an encoder on Party A's half-images using SimCLR [Chen et al., 2020]. Given two stochastic augmentations t_1, t_2 of the same local view, the encoder minimizes the NT-Xent objective:

$$\mathcal{L}_{\mathrm{SimCLR}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp\big(\mathrm{sim}(e(t_1(x_i)), e(t_2(x_i)))/\tau\big)}{\sum_{k\neq i} \exp\big(\mathrm{sim}(e(t_1(x_i)), e(t_2(x_k)))/\tau\big)}, \tag{13}$$

where B is the batch size, τ is the temperature, and $\mathrm{sim}(\cdot,\cdot)$ is cosine similarity. The encoder is then refined on $\mathcal{D}_L$ using supervised contrastive learning [Khosla et al., 2020]:

$$\mathcal{L}_{\mathrm{SupCon}} = -\sum_{i\in\mathcal{D}_L} \frac{1}{|\mathcal{P}(i)|} \sum_{p\in\mathcal{P}(i)} \log \frac{\exp(\mathrm{sim}(e(u_i), e(u_p))/\tau)}{\sum_{a\neq i} \exp(\mathrm{sim}(e(u_i), e(u_a))/\tau)}, \tag{14}$$

where $\mathcal{P}(i)$ denotes the set of labeled positives sharing the same class as sample i. A self-training pass generates pseudo-labels, after which an over-specified Gaussian mixture model is fit in the L2-normalized embedding space. Components are merged into K_c clusters via auxiliary-label prototypes, and GMM responsibilities provide confidence scores $\hat{q}$.

RGB vision: CIFAR-10. CIFAR-10 is harder because a 16-column RGB half-view contains limited semantic context. We use contrastive pretraining followed by a FixMatch-style pipeline [Sohn et al., 2020]. A linear probe trained on $\mathcal{D}_L$ initializes the classifier, and an EMA teacher generates high-confidence pseudo-labels. The FixMatch objective is

$$\mathcal{L}_{\mathrm{fix}} = \frac{1}{\mu B}\sum_{u} \mathbb{1}\Big[\max_y p_\theta(y \mid a_w(u)) \geq \tau_c\Big]\, H(\hat{y}_u,\ p_\theta(y \mid a_s(u)))\,, \tag{15}$$

where a_w and a_s denote weak and strong augmentations, $\hat{y}_u$ is the teacher pseudo-label, μ is the unlabeled batch multiplier, and τ_c is the confidence threshold. A selection step chooses between teacher logits and GMM-based clusters using the auxiliary labeled subset.

Continuous tabular data: UCI-HAR. For UCI-HAR, we use a tabular FixMatch pipeline with an EMA student–teacher MLP. The model uses inverse-frequency class weighting estimated from $\mathcal{D}_L$. Cluster IDs are obtained from the teacher's hard predictions, and confidence scores are the maximum softmax probabilities.

Discrete tabular data: UCI-Mushroom. For UCI-Mushroom, we use a Bernoulli-mixture-based pipeline tailored to binary and one-hot features. The pipeline combines deterministic annealing EM initialization, pseudo-label refinement rounds, graph-based smoothing, and spread-based post-refinement.

Attack-oriented clustering: UCI-Bank. For UCI-Bank, direct binary clustering can yield insufficient donor diversity. We therefore use $K_c = 16$ unsupervised GMM clusters on Party A's high-MI feature block. This setting is attack-oriented: the goal is to provide geometrically diverse donor pools for Phase II rather than maximize direct binary label-alignment.

B Model and Training Details

MNIST and Fashion-MNIST. Each party uses a lightweight local encoder over its half-image view. The server concatenates party embeddings and applies a two-layer MLP head with hidden dimension 32.

CIFAR-10. Each party uses a ResNet-based bottom model over its half-image view. The server applies a concatenation-based fusion head. RandAugment is used during representation learning and training where applicable.

UCI-Mushroom. UCI-Mushroom uses a tabular MLP-based VFL architecture over the MI-partitioned feature views.

UCI-HAR. UCI-HAR uses an asymmetric tabular VFL architecture. The adversarial-side embedding is 96-dimensional, while the passive honest-side embedding is 8-dimensional.

UCI-Bank. UCI-Bank also uses an asymmetric tabular architecture. The adversarial-side embedding is 96-dimensional, and the honest-side embedding is 2-dimensional.

Optimization. Unless otherwise stated, all VFL models are trained for 80 epochs using Adam or AdamW. Dataset-specific optimizer settings, batch sizes, confidence thresholds, and augmentation choices are fixed across clean and attacked runs.

C Per-Dataset Cluster Composition

Tables 5–8 report cluster size, majority label, majority count, and purity for the Phase I partitions.

Table 5: MNIST cluster composition ($K_c = 10$).

Cluster	Size	Maj. Label	Maj. Count	Purity
0	7,193	0	5,599	0.778
1	6,181	1	6,094	0.986
2	7,400	2	5,473	0.740
3	6,592	3	5,505	0.835
4	4,681	4	4,411	0.942
5	4,985	5	4,558	0.914
6	4,562	6	4,297	0.942
7	5,344	7	5,208	0.975
8	5,605	8	5,249	0.936
9	7,457	9	5,536	0.742

Table 6: Fashion-MNIST cluster composition ($K_c = 10$).

Cluster	Size	Maj. Label	Maj. Count	Purity
0	6,523	0	4,768	0.731
1	5,945	1	5,672	0.954
2	7,360	2	3,810	0.518
3	7,613	3	4,869	0.640
4	5,301	4	2,978	0.562
5	7,498	5	5,802	0.774
6	4,117	6	958	0.233
7	3,984	7	3,841	0.964
8	5,201	8	5,145	0.989
9	6,458	9	5,487	0.850

Table 7: CIFAR-10 cluster composition ($K_c = 10$).

Cluster	Size	Maj. Label	Maj. Count	Purity
0	4,988	0	4,102	0.822
1	4,962	1	4,508	0.908
2	4,678	2	3,169	0.678
3	4,565	3	2,751	0.603
4	5,149	4	3,641	0.707
5	4,796	5	3,128	0.652
6	5,620	6	4,138	0.736
7	4,968	7	4,029	0.811
8	5,110	8	4,464	0.874
9	5,164	9	4,437	0.859

Table 8: UCI-HAR cluster composition ($K_c = 6$).

Cluster	Size	Maj. Label	Maj. Count	Purity
0	1,925	5	1,870	0.971
1	1,722	3	1,333	0.774
2	1,951	4	1,514	0.776
3	1,353	2	1,284	0.949
4	1,742	1	1,452	0.834
5	1,606	0	1,520	0.946

D Swap Strategy Details and Attack Ablations

D.1 Swap Strategy Implementation

Optimal Topk. The main CCVS strategy first solves the maximum-distance derangement over cluster centroids. The donor pool may then be extended from the single deranged target to the top-k farthest clusters. Donors are selected from the high-confidence core of this pool using hardest-negative assignment.

Greedy diversity cycling. To avoid repeatedly selecting the same high-contrast donor, greedy diversity cycling marks selected donors as used until the confidence core is exhausted. Only after all available core donors have been used does the algorithm allow repetition. This preserves both local hardness and batch-level diversity.

Class-aware UCI-Bank variant. For UCI-Bank, $K_c = 16$ clusters are optimized for geometric diversity rather than direct label alignment. The class-aware optimal variant uses the auxiliary classifier to select donor regions with opposite-class tendency while preserving the feature-rearrangement constraint.

E Reference Integrity and Adaptive Defense Considerations

Reference integrity. RGAR assumes that the reference set is clean or independently verifiable at calibration time. If an adversary controls the passive party before calibration and can poison $\mathcal{R}$ at submission time, RGAR may learn corrupted prototypes and reconstructors. Possible mitigations include collecting reference features before deployment, using TEE-backed feature extraction, requiring signed audit records, using frozen reference embeddings rather than raw features, and applying robust prototype or outlier filtering.

Embedding-only reference setting. In deployments where raw reference features cannot be shared, RGAR can be implemented over frozen clean reference embeddings. This preserves the calibration role of $\mathcal{R}$ while reducing raw-feature exposure. A fully privacy-preserving implementation using secure enclaves, differential privacy, or cryptographic computation is left for future work.

Adaptive attacks. A patient adversary could avoid reference-like regions, poison only non-reference samples, or introduce slow temporal drift to evade thresholds. RGAR partially addresses this through joint prototype deviation, temporal evidence accumulation, delayed attribution, and revocable trust weights. A full adaptive evaluation remains future work.

F RGAR Algorithm and Diagnostics

Algorithm 2 RGAR: Reference-Guided Attribution and Reconstruction

Require: Reference set $\mathcal{R}$; thresholds $\tau_{\text{pair}}, \tau_{\text{global}}$; burn-in window W; blend weights α, β.
Fit $\{\mathbf{p}_c^A, \mathbf{p}_c^B, \mathbf{p}_c^{\text{int}}, \Sigma_c^A, \Sigma_c^B\}$ on $(h_i^A, h_i^B, y_i)_{i\in\mathcal{R}}$.
Train g_θ on $\mathcal{R}$ to minimize $\mathcal{L}_{\text{recon}}$; freeze g_θ.
Initialize $\rho_A = 1$, $\rho_B = 1$, `attr`=`False`, buffer $\mathcal{E} = \emptyset$.
for each training epoch **do**
  for each mini-batch $\mathcal{B}$ **do**
    Compute s_i^{proto}, e_i^A, and e_i^B via Eqs. (10)–(11).
    Update EMAs; add $e_i^A - e_i^B$ to $\mathcal{E}$ if $s_i^{\text{proto}} > \tau_{\text{pair}}$.
    for each $i \in \mathcal{B}$ **do**
      Form $\hat{h}_i^A$ via Eq. (12).
      $\tilde{h}_i^A \leftarrow (1 - w_i)h_i^A + w_i\hat{h}_i^A$.
    end for
    Forward $(\rho_A\tilde{h}_i^A, \rho_B h_i^B)$ to the head model.
  end for
  if epoch $\geq W$ **and** $\text{mean}(\mathcal{E}) > \tau_{\text{global}}$ **then**
    `attr`$\leftarrow$`True`; $\rho_A \leftarrow \max(\rho_{\min}, \rho_A\gamma)$.
  else if $\text{mean}(\mathcal{E}) < -\tau_{\text{global}}$ **then**
    Revoke attribution; $\rho_A \leftarrow \min(1, \rho_A + \delta)$.
  end if
  Clear $\mathcal{E}$.
end for

F.1 Sensitivity to Top-k Donor Scope

Table 9 shows the effect of increasing the top-k donor scope on MNIST. Smaller k restricts donors to maximally distant clusters and produces stronger semantic mismatch. Larger k introduces moderately distant clusters into the donor pool, weakening the attack.

Table 9: Sensitivity to top-k donor scope on MNIST. Clean accuracy is 96.9%. Drop is the absolute reduction from clean accuracy.

k	Attack Acc. (%)	Drop (pp)
1	33.0	63.9
2	34.5	62.4
3	36.3	60.6
5	50.9	46.0
7	61.4	35.5

F.2 Party Count and Auxiliary Label Ablations

Table 10: Ablation studies. Left: post-attack accuracy (%) versus party count K with one adversarial party. Right: post-attack accuracy (%) versus auxiliary label fraction ε in the two-party setting. Lower accuracy indicates a stronger attack.

Table 11: (a) Party count K

K	MNIST	F-MNIST	HAR	Mush.
2	33.0	33.6	15.5	26.3
4	73.8	78.4	81.9	63.7
8	89.2	91.3	93.8	82.1
10	93.1	94.4	95.6	90.7

Table 12: (b) Auxiliary label fraction ε

ε	MNIST	F-MNIST	HAR	Mush.
0.5%	78.2	81.6	87.4	73.1
1%	71.7	74.3	78.3	61.1
2%	63.2	68.9	71.4	51.4
3%	42.3	45.7	65.6	25.0
5%	33.0	33.6	15.5	26.3

G Computational Resources

All experiments were run on a single GPU or CPU. We used two hardware setups: **Setup A**: NVIDIA GeForce RTX 5090 with 32GB VRAM, AMD Ryzen 9 CPU, and 128GB RAM; and **Setup B**: NVIDIA GeForce RTX 4060 Laptop GPU with 8GB VRAM, Intel Core i7 CPU, and 32GB RAM. Setup A was the primary platform, especially for CIFAR-10; Setup B was used for validation and smaller datasets. No distributed or multi-GPU training was used.

Phase I clustering. Phase I is run once per dataset and its outputs are reused for all Phase II attack and defense runs. Table 13 reports approximate single-run wall-clock times on Setup A.

Table 13: Approximate Phase I clustering runtime on Setup A.

Dataset	Pipeline	Runtime
MNIST	SimCLR + SupCon + GMM merge	$\sim$60 min
Fashion-MNIST	SimCLR + SupCon + GMM merge	$\sim$60 min
CIFAR-10	SimCLR + FixMatch + selection	$\sim$90 min
UCI-HAR	Tabular FixMatch	$\sim$20 min
UCI-Mushroom	Bernoulli mixture + refinement	$\sim$10 min
UCI-Bank	GMM ($K_c = 16$)	$\sim$10 min

Phase II attack training. Each attack run trains a fresh VFL model for 80 epochs on the poisoned dataset. For each dataset, we run five swap strategies plus one clean baseline. CIFAR-10 is the most expensive setting: on Setup A, each run takes approximately 1 hour with batch size 1024; on Setup B, the same run takes about 3 hours with batch size 512 due to VRAM limits. Other datasets are substantially cheaper: MNIST and Fashion-MNIST take about 10–15 minutes per run, UCI-HAR 8–12 minutes, UCI-Mushroom 3–5 minutes, and UCI-Bank 15–20 minutes.

Defense evaluation. Each defense condition trains a separate VFL model from scratch on the same poisoned dataset and evaluates on clean test data. RGAR additionally trains the honest-view reconstructor and performs the reference warm-up phase. On Setup A, per-condition runtimes are

approximately 15 minutes for MNIST/Fashion-MNIST, 10 minutes for UCI-HAR, 20 minutes for UCI-Bank, and 1 hour for CIFAR-10. RGAR adds roughly 15–30 minutes for reconstructor training depending on the dataset.

Total compute. The full six-dataset evaluation, including Phase I clustering, Phase II attack runs, and defense evaluation, requires approximately 25–30 GPU-hours on Setup A. On Setup B, the same workload requires approximately 55–65 GPU-hours, mainly due to slower CIFAR-10 training under the smaller VRAM budget. Phase I cluster artifacts are reused across attack and defense experiments, so the clustering cost is incurred only once per dataset.

H Reproducibility Notes

All experiments use fixed dataset splits, fixed vertical partitions, and the same clean VFL training protocol across clean, attacked, and defended runs. For each dataset, Phase I produces the adversary-side cluster assignments $\hat{c}$, confidence scores $\hat{q}$, and cluster statistics used by Phase II. Phase II then constructs a poisoned adversarial view $\widetilde{X}^{(A)}$ by replacing only the adversary's local features; the honest party's features $X^{(B)}$ and server labels y are never modified. All attack strategies are evaluated under the same VFL architecture and training budget, and defense baselines are run on the same poisoned training views to ensure direct comparability. Dataset-specific partitions, clustering pipelines, model architectures, and swap-strategy details are reported in Appendices A–D.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction accurately summarize the attack, defense, assumptions, and scope. The attack claims are supported across six datasets, while RGAR is presented within its stated trusted-reference setting and evaluated on six complete defense runs.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification:

   Answer: [Yes]

   Justification: The paper includes a Limitations section covering the two-party focus, RGAR's trusted-reference assumption, defense evaluation scope, and future work on collaborative multi-party attacks, adaptive adversaries, and reference poisoning.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is primarily empirical and methodological. It defines the threat model, attack objective, CCVS mechanism, and RGAR procedure formally, but does not state theoretical theorems and so doesn't provide formal proof claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper provides the datasets, vertical partitioning rules, model architectures, training protocol, Phase I clustering pipelines, Phase II swap strategies, defense baselines, evaluation metrics, and reproducibility notes. Appendix details further specify dataset-specific partitioning, clustering procedures, swap implementation, and ablation settings needed to reproduce the main attack and defense results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We have provided anonymized code in the supplemental material, including scripts and configuration files to reproduce the main CCVS attack results, defense baselines, RGAR evaluation, and ablations. The supplement also includes instructions for dataset preparation, vertical partitioning, intermediate Phase I artifacts, and commands needed to reproduce the reported experimental results. DO

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The main paper describes the datasets, vertical partitioning, model families, attack strategies, defense baselines, and evaluation metrics. Full training details, including dataset-specific partitions, model architectures, optimizer choices, clustering pipelines, swap settings, and ablation configurations, are provided in the appendix and supplemental code.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: The current submission reports point estimates under fixed experimental protocols and fixed attack settings, but does not yet include error bars, confidence intervals, or statistical significance tests. We mitigate this by using consistent dataset splits, fixed vertical partitions, and identical attack/defense settings across methods, but multi-seed variance analysis is left for future work due to computational constraints.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The main paper refers to the appendix including a Computational Resources section specifying the hardware setups, GPU/CPU memory, single-GPU or CPU execution, approximate Phase I clustering runtimes, Phase II per-run attack training times, defense evaluation runtimes, and total compute estimates. The paper also states that no distributed or multi-GPU training is used and that Phase I artifacts are reused across attack and defense runs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. The work uses public benchmark datasets, does not involve human subjects or personally identifiable information, and is presented as security research with a defensive purpose. The paper discusses the vulnerability of VFL to Byzantine attacks and proposes RGAR as a mitigation.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses both positive and negative societal impacts. Positively, the work helps identify and mitigate availability risks in privacy-preserving VFL systems used in high-stakes domains such as finance and healthcare. Negatively, the proposed attack is dual-use and could be misused to degrade collaborative models; the paper mitigates this risk by clearly bounding the threat model, discussing limitations, and proposing RGAR as a defense.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: The paper does not release high-risk pretrained models or scraped datasets, but the attack code is dual-use. We mitigate this by releasing only research code for public benchmark settings, documenting the threat model and limitations, and presenting RGAR as a corresponding defense to support responsible security evaluation.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the original sources for all existing datasets and model components used in the experiments. The supplemental code documents dataset access through the original providers and does not repackage restricted assets; applicable licenses and terms of use are respected through those original sources.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces anonymized research code for reproducing the CCVS attack, defense baselines, RGAR evaluation, and ablations. The supplemental material documents the experimental pipeline, dataset preparation, configuration files needed to reproduce the main results. Intermediate clusters could not be uploaded due to size limit. All the scripts and commands needed for reproduction have been uploaded with the supplementary material.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.

- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing, human-subject experiments, participant studies, or newly collected human data. All experiments use public benchmark datasets.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve human-subject research, crowdsourcing, or interaction with study participants. Therefore, IRB or equivalent approval is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: The core methodology, experiments, attack design, defense design, and evaluation do not use LLMs as an important, original, or non-standard component. Any LLM assistance was limited to writing, editing, or formatting support and did not affect the scientific method or experimental results.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.

- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.